

17 — Due simboli, un solo angolo

La domanda a cui risponde: due funzioni diverse chiedono lo stesso posto e lo stesso simbolo sullo schermo. Come si decide chi lo tiene? E, più in generale, quando due stati riguardano lo stesso oggetto — una carta — conviene un campo con più valori o due campi indipendenti?

Il fatto

La collezione aveva già una **lista dei desideri**: carte che *non* hai e vorresti. Sulla miniatura sono contrassegnate da un badge viola con una stella — 2 significa “ne vorrei due”.

È arrivata la richiesta dei **preferiti**: segnare le carte che ti piacciono, per ritrovarle senza cercarle. Il gesto naturale, in qualunque app, è una stellina da toccare. Ed è esattamente il simbolo già occupato.

Due stelle sulla stessa miniatura — una che dice “ne vorrei due” e una che dice “mi piace” — sarebbero indistinguibili proprio nella condizione in cui servono: l'occhio che scorre una griglia di sessanta carte.

Prima decisione: separare i canali, non solo i simboli

La soluzione non è stata scegliere “quale delle due merita la stella”, ma accorgersi che i due segni si distinguono su **quattro canali indipendenti**, e usarli tutti:

	Desiderio	Preferito
forma	stella □	cuore □
posizione	alto a destra	alto a sinistra
colore	viola (--colore-proxy)	rosso (--colore-preferito)
natura	<i>badge</i> , non si tocca	<i>pulsante</i> , si tocca

Il quarto è il più importante e il meno visibile: il badge dei desideri **informa**, il cuore **comanda**. Se avessimo dato ai preferiti una stella grigia da accendere, l'utente avrebbe dovuto imparare che una stella si tocca e l'altra no.

Sui colori vale una regola che il progetto seguiva già: il viola in quest'app significa “*questa carta non è nella scatola*” — lo usano i proxy da stampare e i desideri. Un preferito è l'esatto contrario, una carta che hai. Dargli il viola avrebbe rotto un significato già insegnato all'utente.

E il colore da solo non basta mai: acceso il cuore è **pieno**, spento è un **contorno**. La differenza si vede anche in bianco e nero, cioè anche a chi non distingue rosso e grigio.

```
// src/ui/griglia-collezione/griglia-collezione.js
fill={`${accesso ? 'currentColor' : 'none'}`} stroke="currentColor"
```

Seconda decisione: due campi, non un campo con tre valori

Sul modello dati la tentazione era di allargare quello che c'era. La riga di collezione aveva già:

```
{ id: 'sv08:118', idSet: 'sv08', numero: '118', quantita: 2, desiderata: true }
```

Si poteva fare stato: 'posseduta' | 'desiderata' | 'preferita'. **Sarebbe stato sbagliato**, e il modo di accorgersene è una domanda sola: *le due cose si escludono davvero?*

- Posseduta / desiderata **sì**: o ce l'hai o non ce l'hai. Un solo campo.
- Posseduta / preferita **no**: un preferito è una carta posseduta, con in più un giudizio.

Un `enum` per stati che possono coesistere è un errore che si paga più tardi: il giorno in cui serve “preferita e in tre copie” bisogna disfare il campo e migrare i dati. Due campi indipendenti costano un `if` in più e non hanno quel giorno.

```
// Quello che si scrive davvero, in src/data/collezione.js
{ ...riga, ...(preferita ? { preferita: true } : {}) }
```

Nota il pattern, già usato per desiderata: **il campo si scrive solo quando è vero**. Niente `preferita: false`. Assente e falso direbbero la stessa cosa in due modi, e la differenza sporcherebbe l'export e i confronti. In IndexedDB non serve nessuna migrazione: gli object store conservano oggetti liberi, e le righe vecchie semplicemente non hanno il campo.

Rispetto a Java: qui non c'è uno schema che rifiuta le colonne sconosciute, né una classe che dichiara i campi. La forma dei dati è un **accordo fra chi scrive e chi legge**, e nessun compilatore lo sorveglia. È lo stesso rischio del documento 08 — e la ragione per cui `impostaPreferita()` ha un JSDoc che dice cosa scrive, e un test che lo prova.

Il difetto che il campo nuovo ha creato

Aggiungere un campo a una riga scritta da tre funzioni diverse ne rompe una in silenzio. `impostaQuantita()` non aggiorna la riga: la **riscrive da capo**.

```
// Prima. Sembra innocuo, e per mesi lo è stato.
await scrivi(STORE_COLLEZIONE, {
  id, idSet, numero: String(numero),
  quantita: Math.floor(quantita),
  aggiornatoIl: new Date().toISOString(),
});
```

Con `preferita` in gioco, toccare **+** su una carta preferita la toglieva dai preferiti. Nessun errore, nessun messaggio: il cuore si spegneva al ridisegno successivo, cioè abbastanza tardi da non collegarlo al gesto.

La correzione è rileggere e riportare il campo:

```
const esistente = await leggi(STORE_COLLEZIONE, id);
await scrivi(STORE_COLLEZIONE, {
  ...,
  ...(esistente?.preferita ? { preferita: true } : {}),
});
```

La regola generale: ogni scrittura che ricostruisce un record invece di modificarlo è un punto in cui i campi aggiunti dopo si perdono. Sono i punti da rivedere per primi quando si allarga un modello dati.

Terza decisione: la vista è la stessa griglia

“Preferiti” è una voce della tab bar, quindi una vista. La strada breve sarebbe stata un componente nuovo che disegna un elenco di preferiti. Sarebbe stato il doppio del comportamento da tenere allineato per sempre: ricerca, filtri, raggruppamento per serie, prezzi, lazy-load delle immagini.

Invece è **la stessa** `<griglia-collezione>` con un filtro che l'utente non può togliere:

```
// src/app/app.js
grigliaPreferiti.titolo = 'I preferiti';
grigliaPreferiti.filtriFissi = { preferito: 'solo' };
```

Il punto sottile è *dove* mettere quel filtro. Dentro `#filtri`, insieme a quelli scelti dall'utente, non funziona: il pulsante “azzera filtri” li riporta a `FILTRI_VUOTI` e la vista Preferiti avrebbe cominciato a mostrare tutto. Perciò stanno in un oggetto a parte, e le due cose si sommano solo al momento dell'uso:

```
#effettivi() {
  return { ...this.#filtri, ...this.#fissi };
}
```

Un componente con due sorgenti di configurazione — quella dell'utente e quella di chi lo ospita — è lo stesso schema di un `@Input()` Angular che convive con lo stato interno: la differenza è che qui la fusione è esplicita e sta in una riga che si può leggere.

E le voci passate alla griglia dei preferiti sono **tutte**, non solo quelle col cuore: è il filtro fisso a scremarle. Passandole già scremate, i menu a tendina (serie, set, rarità) si ridurrebbero a ciò che è preferito, e non si capirebbe più cosa stanno filtrando.

Un dettaglio di HTML che morde

Il cuore sta sopra la miniatura, che è dentro il pulsante `.apri-carta` che apre la carta a schermo intero. La scrittura istintiva è annidarli lì.

Un `<button>` dentro un `<button>` è HTML non valido. Il parser non segnala niente: chiude il primo pulsante e sposta il secondo fuori, riscrivendo l'albero. Il risultato è un cuore che compare in un punto sbagliato della card, con l'aspetto di un bug del CSS — e ore spese a guardare il foglio di stile.

La soluzione è renderlo **fratello**, e sovrapporlo con il posizionamento:

```
.carta-griglia { position: relative; } /* il riferimento */
.carta-griglia .cuore { position: absolute; top: 6px; left: 6px; z-index: 2; }
```

Il tocco che risponde subito

Il cuore si accende **prima** che il database confermi:

```
cuore.classList.toggle('acceso', acceso);
cuore.setAttribute('aria-pressed', String(acceso));
this.dispatchEvent(new CustomEvent('preferita-cambiata', { ... }));
```

Si chiama *optimistic UI*. Su telefono il giro completo — scrittura in IndexedDB, riletture della collezione, ridisegno — dura abbastanza da far sembrare che il tocco non sia stato raccolto. La verità arriva comunque subito dopo: se la scrittura fallisse, il ridisegno rimetterebbe il cuore com'era.

`aria-pressed` non è decorazione: è ciò che rende il cuore un **interruttore** per chi usa uno screen reader, che altrimenti sentirebbe solo “pulsante”.

Esercizi

1. **Il campo che sparisce.** `impostaDesiderio()` riscrive la riga da capo, come faceva `impostaQuantita()`, e *non* riporta `preferita`. Perché in quel caso è la cosa giusta? (Suggerimento: cosa vuol dire, per una carta, passare da posseduta a desiderata?)
2. **Il terzo stato.** Immagina che arrivi la richiesta “segna le carte doppie da scambiare”. Ti servirebbe un campo nuovo o basta un valore in più su uno esistente? Applica la domanda della sezione “due campi, non un campo con tre valori” e giustifica la risposta in due righe.
3. **Filtri fissi.** In `griglia-collezione.js`, `#soloTue()` impedisce di mostrare le carte mancanti quando è attivo il filtro `desideri` o quello `preferiti`. Togli quella condizione e apri i Preferiti con “mostra anche le carte che mi mancano”: cosa compare, e perché è una risposta a una domanda che nessuno ha fatto?
4. **Contrasto.** Il cuore acceso usa `--colore-preferito` su sfondo bianco al confine dell'immagine della carta. Apri gli strumenti di sviluppo e verifica il contrasto sopra un'illustrazione chiara. Cosa cambieresti nel CSS — l'opacità dello sfondo, un bordo, un'ombra? Provalo.